ENTERPRISE JAVA
[25MCA201]

By,

Raksha V Shetty

Assistant Professor

Department of MCA

NMAMIT, Nitte

Books

Text Books

- Herbert Schildt : Java: The Complete Reference, Eleventh Edition, Tata McGraw Hill., 2019
- Java Server Programming Java EE 7 (J2EE 1.7), Black Book, Dreamtech press 2014

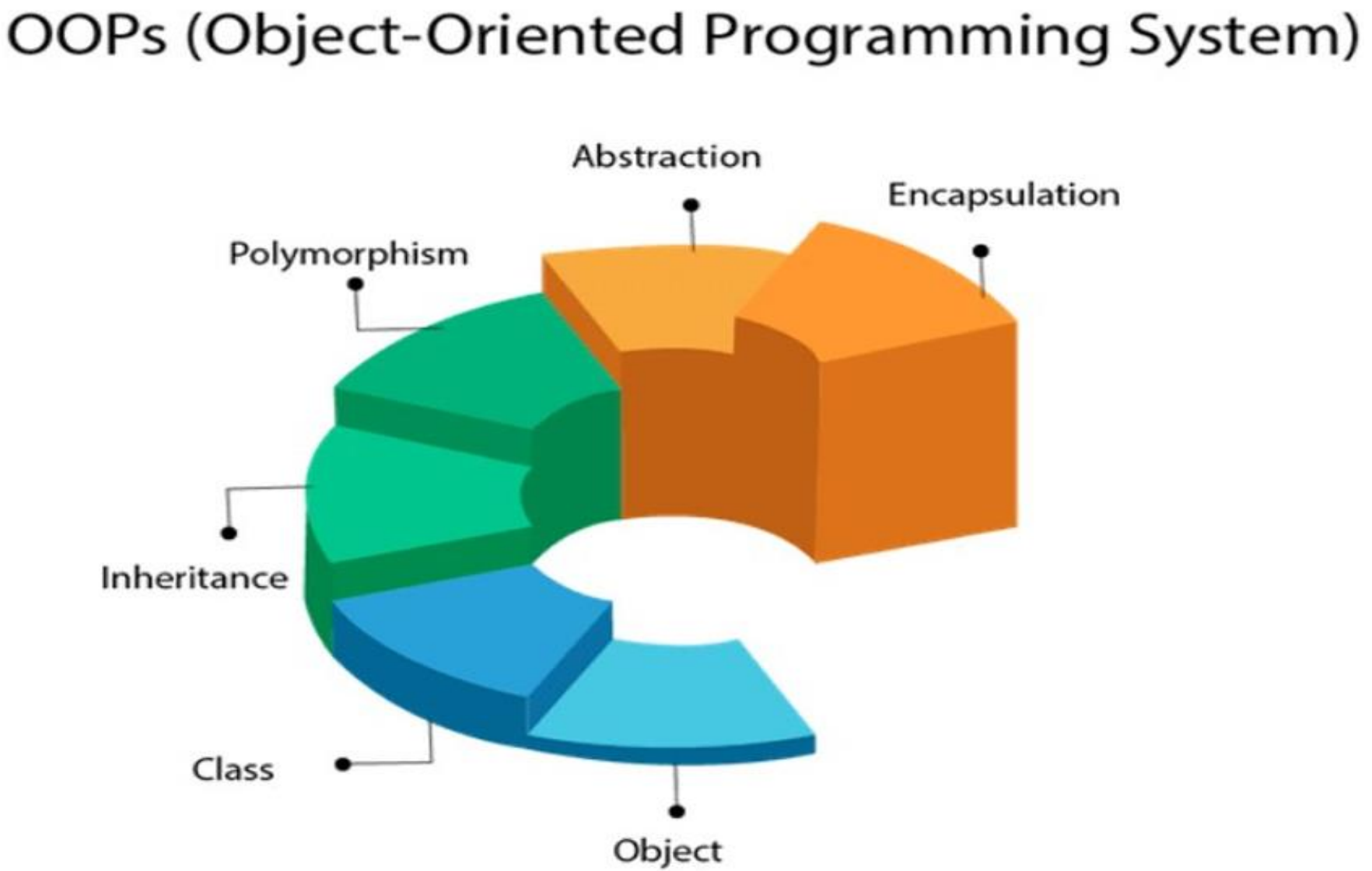
Reference Books

- Dr. Donald Doherty and Rick Leinecker : JavaBeans Unleashed
- James Goodwill : Developing Java Servlets
- Karl Avedal, Danny Ayers : Professional JSP
- Steven Holzner : Java 2 Black Book
- Ed Roman : Mastering Enterprise JavaBeans
- Jim Keogh : The Complete Reference J2EE, Tata McGraw Hill, 2008.

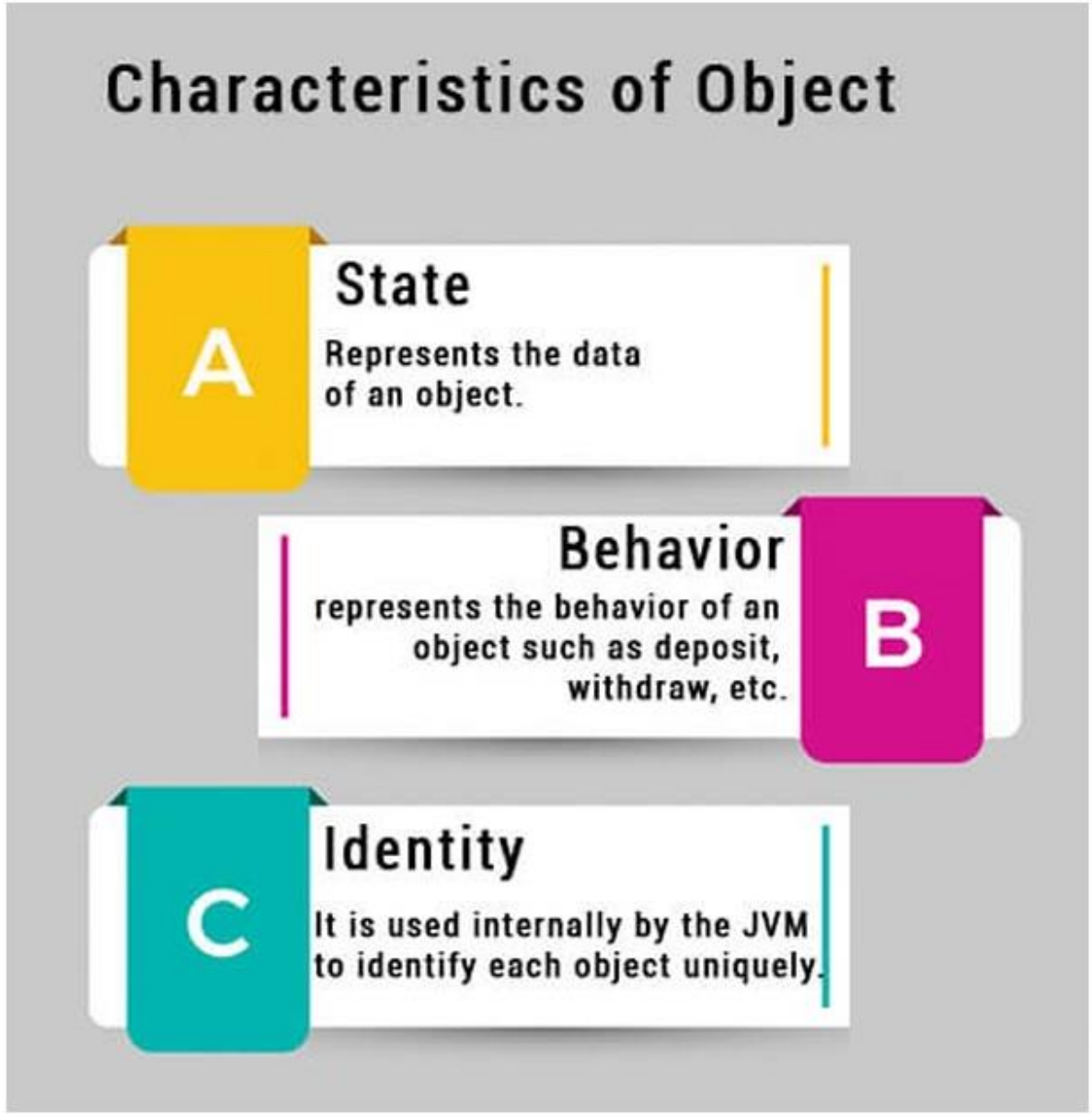
Unit 1

Introducing Classes

Introducing classes



Class Fundamentals



For Example,

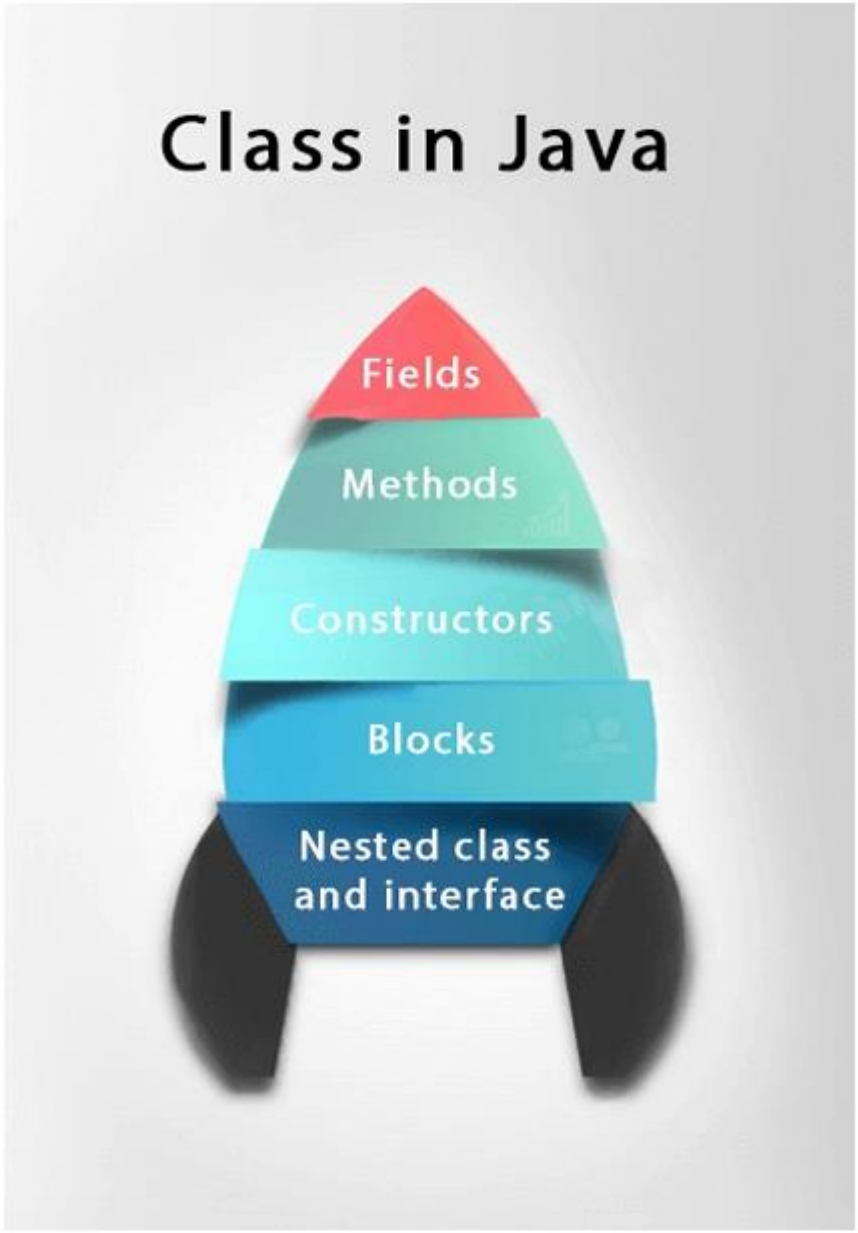
Pen is an object.

Its name is Reynolds;

color is white, known as its state.

It is used to write, so writing is its behavior.

Class Fundamentals



Syntax to declare a class:

```
class <class_name>
{
    field;
    method;

    public static void main(String args[])
   {
    //Creating an object or instance
    //accessing member through reference
      variable
  }
}
```

Class Fundamentals

Instance variable in Java

A variable which is created inside the class but outside the method is known as an instance variable. Instance variable doesn't get memory at compile time. It gets memory at runtime when an object or instance is created. That is why it is known as an instance variable.

Method in Java

In Java, a method is like a function which is used to expose the behaviour of an object.
Advantage of Method

* Code Reusability
* Code Optimization

new keyword in Java

The new keyword is used to allocate memory at runtime. All objects get memory in Heap memory area.

Syntax: Class_name object_name = new class_name()

Ways to initialize object

1) Initialization through reference

Initializing an object means storing data into the object.

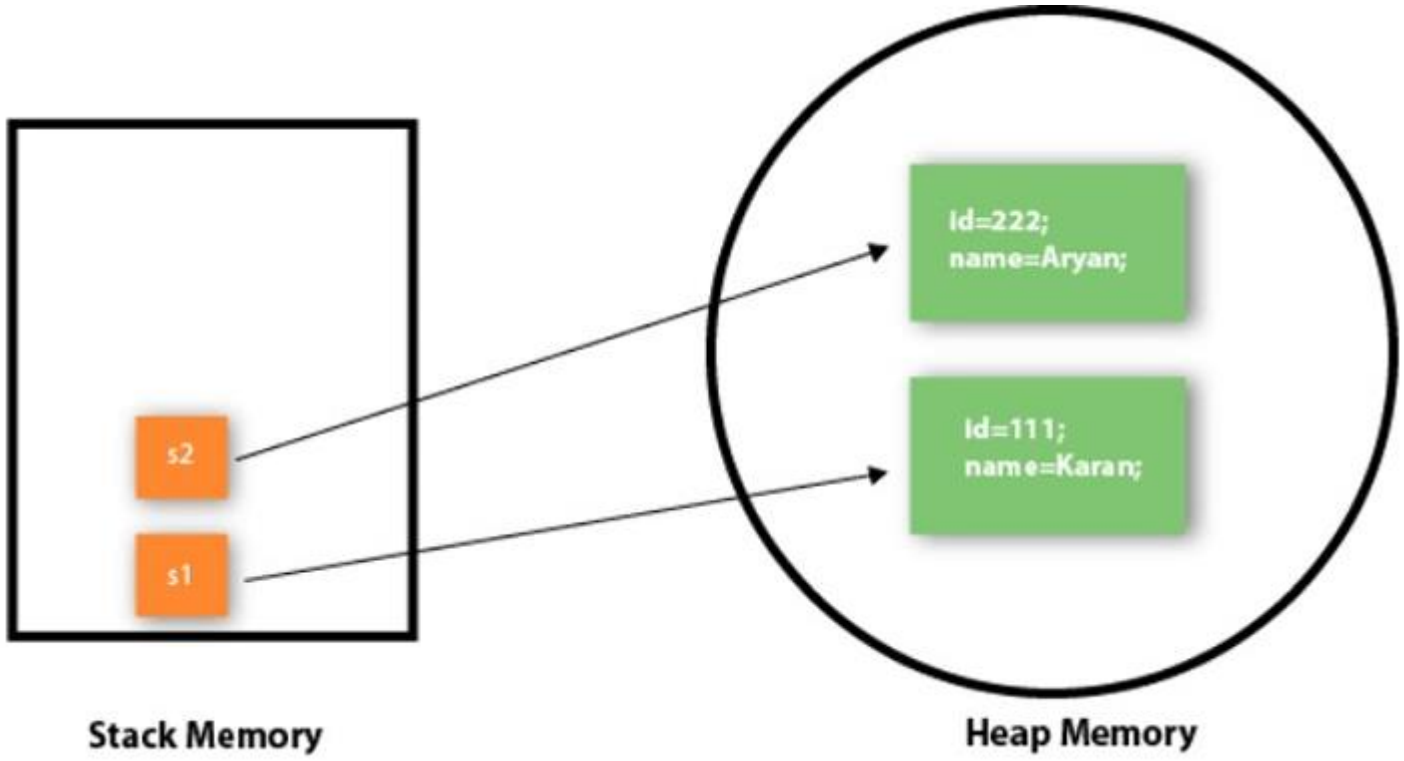
```
class Student{
 int id;
 String name;
}
class TestStudent3{
 public static void main(String args[]){
  //Creating objects
  Student s1=new Student();
  Student s2=new Student();
  //Initializing objects
  s1.id=101;
  s1.name="Sonoo";
  s2.id=102;
  s2.name="Amit";
  //Printing data
  System.out.println(s1.id+" "+s1.name);
  System.out.println(s2.id+" "+s2.name);
 }
}
```

Ways to initialize object

2) Initialization through method

```
class Student{
 int rollno;
 String name;
 void insertRecord(int r, String n){
  rollno=r;
  name=n;
 }
 void displayInformation(){System.out.println(rollno+" "+name);}
}
```

```
class TestStudent4{
 public static void main(String args[]){
  Student s1=new Student();
  Student s2=new Student();
  s1.insertRecord(111,"Karan");
  s2.insertRecord(222,"Aryan");
  s1.displayInformation();
  s2.displayInformation();
 }
}
```



Object gets the memory in heap memory area.
The reference variable refers to the object allocated in the heap memory area.
Here, s1 and s2 both are reference variables that refer to the objects allocated in memory

Ways to initialize object

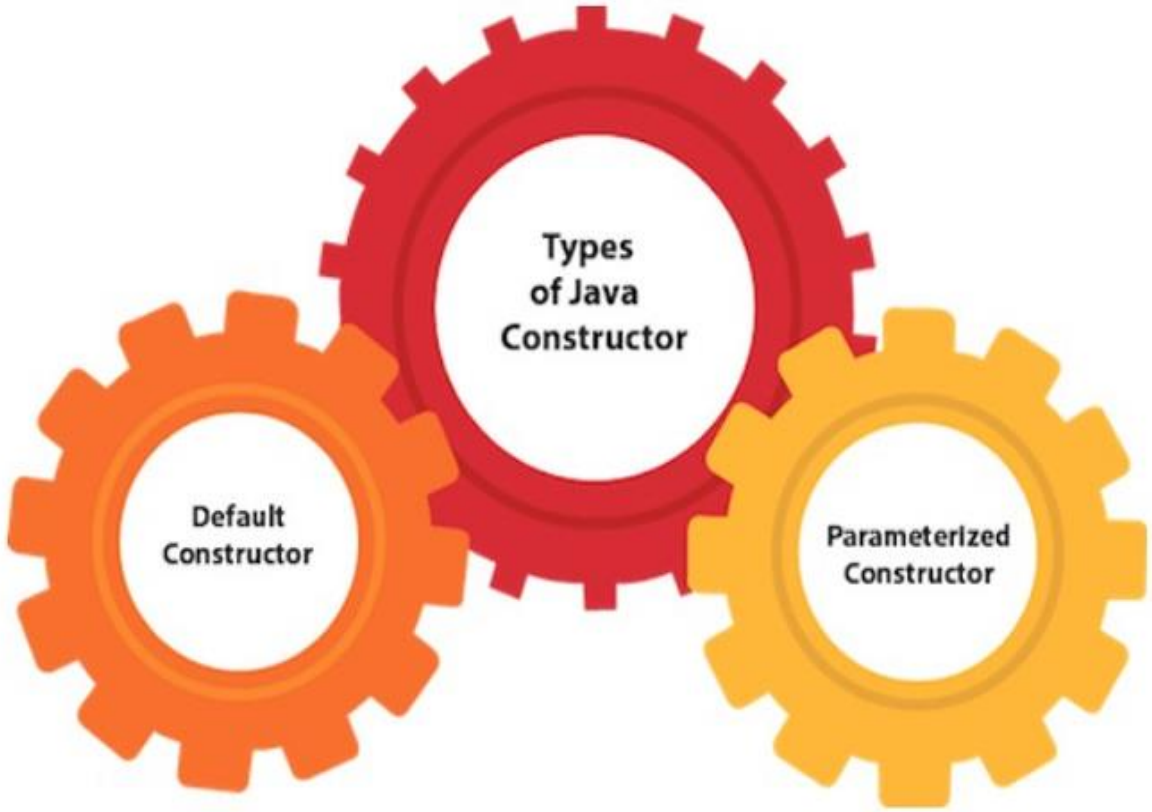
3) Initialization through Constructor

- A constructor is a block of codes similar to the method.
- It is called when an instance of the class is created.
- At the time of calling constructor, memory for the object is allocated in the memory.
- It is a special type of method which is used to initialize the object.
- Every time an object is created using the new() keyword, at least one constructor is called.

Rules for creating Java constructor

- Constructor name must be the same as its class name
- A Constructor must have no explicit return type
- A Java constructor cannot be abstract, static, final, and synchronized

Types of Java constructors



A constructor is called "Default Constructor" (no-arg constructor) when it doesn't have any parameter.

A constructor which has a specific number of parameters is called a parameterized constructor.

Example of default constructor that displays the default values

```
class Student3{
int id;
String name;
//method to display the value of id and name
void display(){System.out.println(id+" "+name);}
```

```
public static void main(String args[]){
//creating objects
Student3 s1=new Student3();
Student3 s2=new Student3();
//displaying values of the object
s1.display();
s2.display();
}
}
```

Predict the Output

Example of parameterized constructor

```
class Student4{
    int id;
    String name;
    //creating a parameterized constructor
    Student4(int i,String n){
    id = i;
    name = n;
    }
    //method to display the values
    void display(){System.out.println(id+" "+name);}
```

```
    public static void main(String args[]){
    //creating objects and passing values
    Student4 s1 = new Student4(111,"Karan");
    Student4 s2 = new Student4(222,"Aryan");
    //calling method to display the values of object
    s1.display();
    s2.display();
   }
}
```

Constructor & Method Overloading

- **Constructor overloading** in Java is a technique of having more than one constructor with different parameter lists (Either in number of parameters or in type of parameters)
- They are arranged in a way that each constructor performs a different task.
- They are differentiated by the compiler by the number of parameters in the list and their types.
- If a class has multiple methods having same name but different in parameters, it is known as **Method Overloading**.
- Method overloading increases the readability of the program.
- Method overloading is not possible by changing the return type of the method only because of ambiguity.

Example of Constructor Overloading

```
class Student5{
    int id;
    String name;
    int age;
    //creating two arg constructor
    Student5(int i,String n){
    id = i;
    name = n;
    }
    //creating three arg constructor
    Student5(int i,String n,int a){
    id = i;
    name = n;
    age=a;
    }
    void display(){System.out.println(id+" "+name+" "+age);}

    public static void main(String args[]){
    Student5 s1 = new Student5(111,"Karan");
    Student5 s2 = new Student5(222,"Aryan",25);
    s1.display();
    s2.display();
   }
}
```

Example of Method Overloading

```
class Adder{
static int add(int a, int b){return a+b;}
static double add(double a, double b){return a+b;}
}
class TestOverloading2{
public static void main(String[] args){
System.out.println(Adder.add(11,11));
System.out.println(Adder.add(12.3,12.6));
}}
```

Q) Why Method Overloading is not possible by changing the return type of method only?

In java, method overloading is not possible by changing the return type of the method only because of *ambiguity*.

```
class Adder{

static int add(int a,int b){return a+b;}

static double add(int a,int b){return a+b;}

}

class TestOverloading3{

public static void main(String[] args){

System.out.println(Adder.add(11,11));//ambiguity

}}
```

Output:

Compile Time Error:
method add(int,int) is already defined in class Adder

Q) Can You overload java main() method?

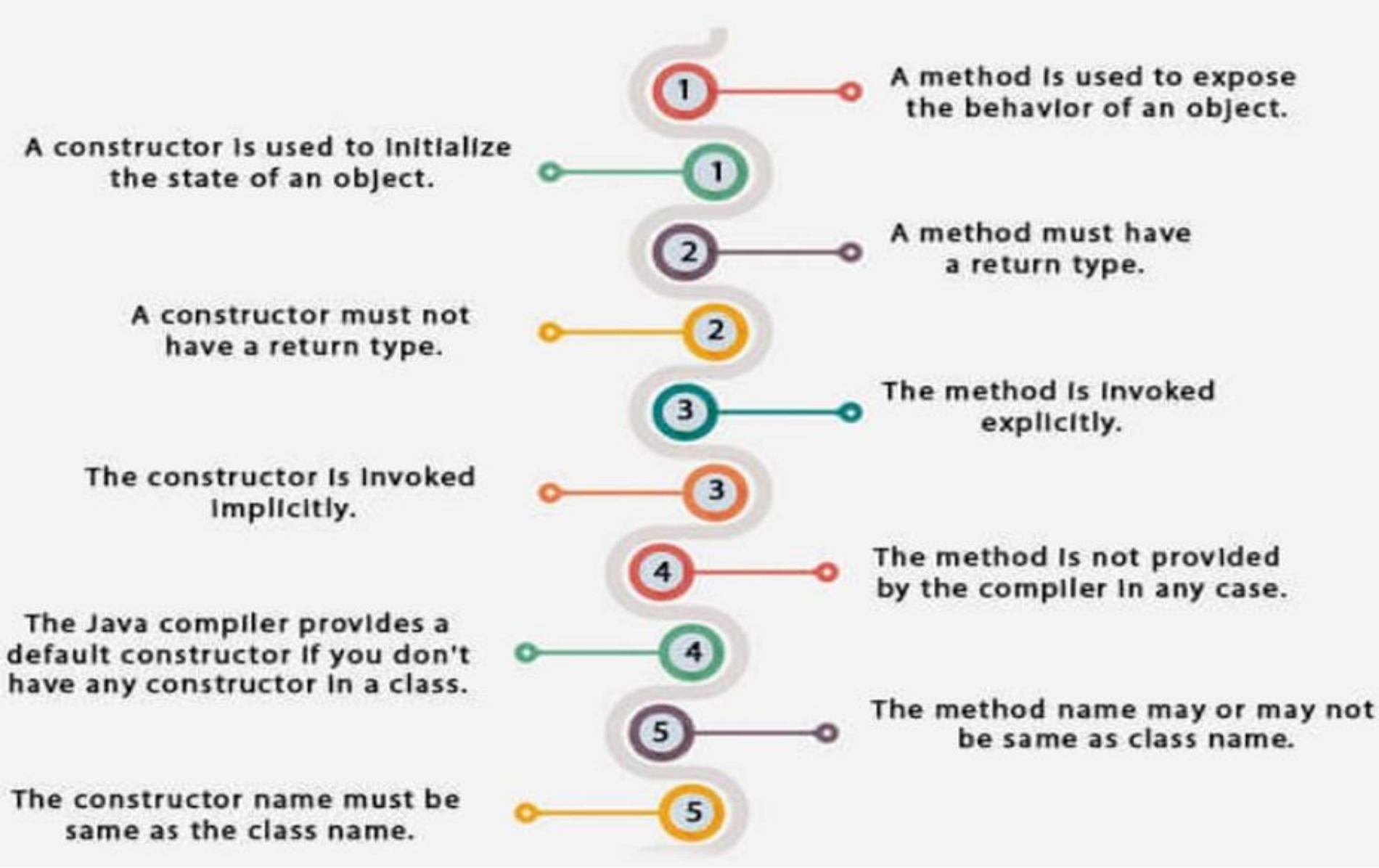
- Yes, by method overloading.
- You can have any number of main methods in a class by method overloading.
- But JVM calls main() method which receives string array as arguments only.

```
class TestOverloading4{
public static void main(String[] args){System.out.println("main with String[]");}
public static void main(String args){System.out.println("main with String");}
public static void main(){System.out.println("main without args");}
}
```

Output:

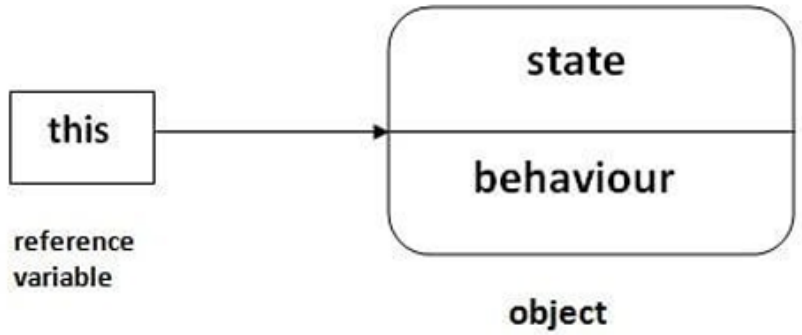
main with String[]

Differences between constructor and method



this keyword

this is a reference variable that refers to the current object.



Usage of this keyword

- this can be used to refer current class instance variable.
- this can be used to invoke current class method (implicitly)
- this() can be used to invoke current class constructor.
- this can be passed as an argument in the method call.
- this can be passed as argument in the constructor call.
- this can be used to return the current class instance from the method.
- If parameters (formal arguments) and instance variable names are same, we can use this keyword to distinguish local variable and instance variable.

1) this: to refer current class instance variable

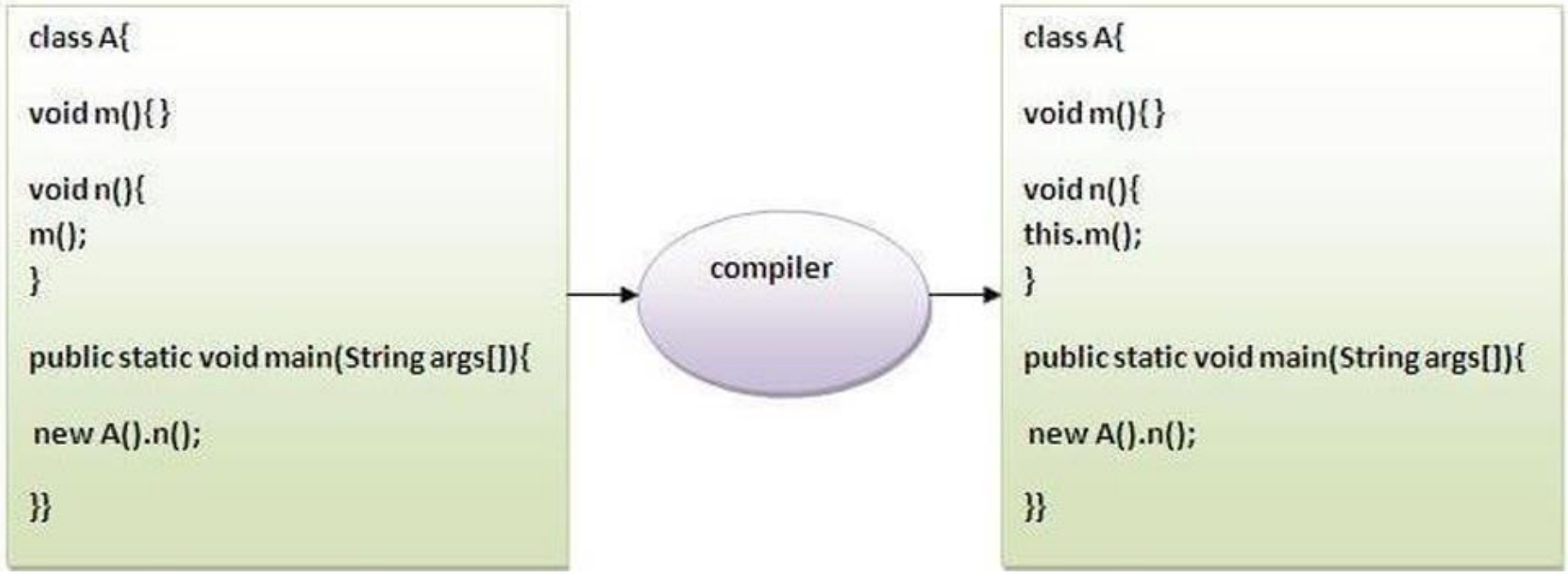
```
class Student{
int rollno;
String name;
float fee;
Student(int rollno,String name,float fee){
this.rollno=rollno;
this.name=name;
this.fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+fee);}
}
```

```
class TestThis2{
public static void main(String args[]){
Student s1=new Student(111,"ankit",5000f);
Student s2=new Student(112,"sumit",6000f);
s1.display();
s2.display();
}}
```

2) this: to invoke current class method

You may invoke the method of the current class by using the this keyword.

If you don't use the this keyword, compiler automatically adds this keyword while invoking the method



2) this: to invoke current class method

```
class A{
void m(){System.out.println("hello m");}
void n(){
System.out.println("hello n");
//m();//same as this.m()
this.m();
}
}
class TestThis4{
public static void main(String args[]){
A a=new A();
a.n();
}}
```

Output:

hello n
hello m

3) this() : to invoke current class constructor

The this() constructor call can be used to invoke the current class constructor.

It is used to reuse the constructor. In other words, it is used for *constructor chaining.*

Calling default constructor from parameterized constructor:

```
class A{
A(){System.out.println("hello a");}
A(int x){
this();
System.out.println(x);
}
}
```

```
class TestThis5{
public static void main(String args[]){
A a=new A(10);
}}
```

Output:

hello a
10

3) this() : to invoke current class constructor

Calling parameterized constructor from default constructor:

```
class A{
A(){
this(5);
System.out.println("hello a");
}
A(int x){
System.out.println(x);
}
}
```

```
class TestThis6{
public static void main(String args[]){
A a=new A();
}}
```

Output:

```
5
hello a
```

3) this() : to invoke current class constructor

Real usage of this() constructor call (Rule: Call to this() must be the first statement in constructor.)

The this() constructor call should be used to reuse the constructor from the constructor.

It maintains the chain between the constructors i.e. it is used for constructor chaining.

```
class Student{
int rollno;
String name,course;
float fee;
Student(int rollno,String name,String course){
this.rollno=rollno;
this.name=name;
this.course=course;
}
Student(int rollno,String name,String course,float fee){
this(rollno,name,course);//reusing constructor
this.fee=fee;
}
void display(){System.out.println(rollno+" "+name+" "+course+" "+fee);}
}
```

```
class TestThis7{
public static void main(String args[]){
Student s1=new Student(111,"ankit","java");
Student s2=new Student(112,"sumit","java",6000f);
s1.display();
s2.display();
}}
```

Output:

```
111 ankit java null
112 sumit java 6000
```

4) this: to pass as an argument in the method

```
  class S2{
  void m(S2 obj){
  System.out.println("object passed is: "+obj);
  }
  void p(){
  m(this);
  }
  public static void main(String args[]){
  S2 s1 = new S2();
  s1.p();
  }
}
```

Output:

object passed is: S2@77459877

5) this: to pass as argument in the constructor call

```
  class B{
  A4 obj;
  B(A4 obj){
    this.obj=obj;
  }
  void display(){
    System.out.println(obj.data);//using data member of A4 class
  }
}

class A4{
  int data=10;
  A4(){
   B b=new B(this);
   b.display();
  }
  public static void main(String args[]){
   A4 a=new A4();
  }
}
```

Output:
10

6) this keyword can be used to return current class instance

```
class A{
A getA(){
return this;
}
void msg(){System.out.println("Hello java");}
}
class Test1{
public static void main(String args[]){
new A().getA().msg();
}
}
```

Output:
Hello java

Introducing Access Control

- There are two types of modifiers in Java: access modifiers and non-access modifiers.
- The access modifiers in Java specifies the accessibility or scope of a field, method, constructor, or class.
- We can change the access level of fields, constructors, methods, and class by applying the access modifier on it.

  There are four types of Java access modifiers:

  Default, Private, Public, Protected

- Non-Access Modifiers are:

  static, abstract, synchronized, native, volatile, transient, etc

Access Modifiers

1. **Private**: The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
2. **Default**: The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected**: The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public**: The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Access Modifiers

Access Modifier	within class	within package	outside package by subclass only	outside package
Private	Y	N	N	N
Default	Y	Y	N	N
Protected	Y	Y	Y	N
Public	Y	Y	Y	Y

Private Access Modifier within class/package

```
class A{
private int data=40;
private void msg(){System.out.println("Hello java");}
}

public class Main{
 public static void main(String args[]){
   A obj=new A();
   System.out.println(obj.data);//Compile Time Error
   obj.msg();//Compile Time Error
   }
}
```

```
class A{
private A(){}//private constructor
void msg(){System.out.println("Hello java");}
}
public class Main{
 public static void main(String args[]){
   A obj=new A();//Compile Time Error
 }
}
```

Output:

```
Main.java:9: error: data has private access in A
      System.out.println(obj.data);//Compile Time Error
                         ^
Main.java:10: error: msg() has private access in A
      obj.msg();//Compile Time Error
         ^
2 errors
```

```
Main.java:7: error: A() has private access in A
       A obj=new A();//Compile Time Error
             ^
1 error
```

Default/Public/Protected Access Modifiers within class/package

```
class A{
A(){}
void defaultmsg(){System.out.println("Default");}
public void publicmsg(){System.out.println("Public");}
protected void protectedtmsg(){System.out.println("Protected");}
}
public class Main{
 public static void main(String args[]){
   A obj=new A();
   obj.defaultmsg();
   obj.publicmsg();
   obj.protectedtmsg();
 }
}
```

Output:

```
Default
Public
protected
```

Nested Classes

- In Java, it is possible to define a class within another class, such classes are known as nested classes.
- They enable you to logically group classes that are only used in one place, thus this increases the use of encapsulation and creates more readable and maintainable code.
- The scope of a nested class is bounded by the scope of its enclosing class.
- A nested class has access to the members, including private members, of the class in which it is nested. But the enclosing class does not have access to the member of the nested class.
- A nested class is also a member of its enclosing class.
- As a member of its enclosing class, a nested class can be declared private, public, protected, or package-private(default).
- Nested classes are divided into two categories:

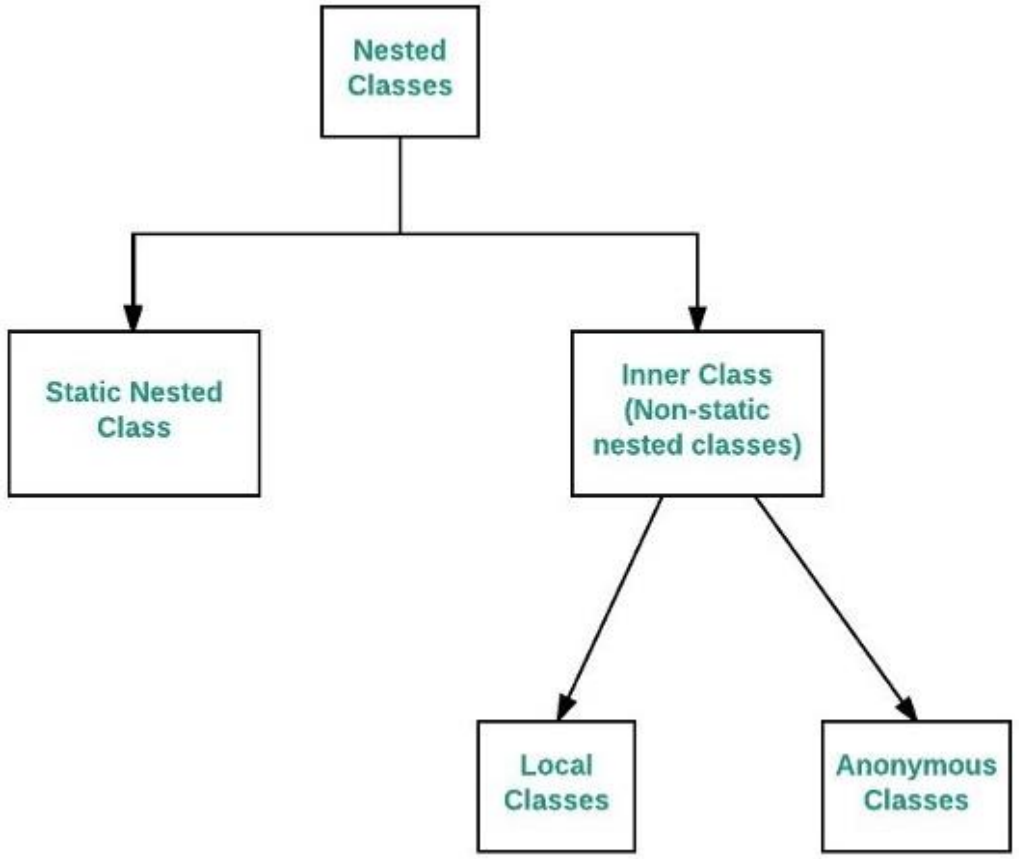
static nested class: Nested classes that are declared static are called static nested classes.

inner class: An inner class is a non-static nested class.

Nested Classes

Syntax:

```
class OuterClass
{
...
    class NestedClass
    {
        ...
    }
}
```



Static Nested Classes

- In the case of normal or regular inner classes, without an outer class object existing, there cannot be an inner class object. i.e., an object of the inner class is always strongly associated with an outer class object.
- But in the case of static nested class, Without an outer class object existing, there may be a static nested class object. i.e., an object of a static nested class is not strongly associated with the outer class object.
- As with class methods and variables, a static nested class is associated with its outer class.
- And like static class methods, a static nested class cannot refer directly to instance variables or methods defined in its enclosing class: it can use them only through an object reference. They are accessed using the enclosing class name.

Static Nested Classes

```
OuterClass.StaticNestedClass
```

For example, to create an object for the static nested class, use this syntax:

```
OuterClass.StaticNestedClass nestedObject =
     new OuterClass.StaticNestedClass();
```

Static Nested Classes

```
// outer class
class OuterClass {
    static int outer_x = 10;
    int outer_y = 20;
    private static int outer_private = 30;
    // static nested class
    static class StaticNestedClass {
        void display()
        {
            System.out.println("outer_x = " + outer_x);
            // can access private static member of outer class
            System.out.println("outer_private = "+ outer_private);
    // The following statement will give compilation error as static nested class
        cannot directly access non-static members
            // System.out.println("outer_y = " + outer_y);

    // Therefore create object of the outer class to access the non-static member
            OuterClass out = new OuterClass();
            System.out.println("outer_y = " + out.outer_y);
        }
    }
}
```

Static Nested Classes

```java
// Driver class
public class StaticNestedClassDemo {
    public static void main(String[] args)
    {
        // accessing a static nested class
    OuterClass.StaticNestedClass nestedObject= new OuterClass.StaticNestedClass();
        nestedObject.display();
    }
}
```

Output

```
outer_x = 10
outer_private = 30
outer_y = 20
```

Inner Classes

- To instantiate an inner class, you must first instantiate the outer class.
- Then, create the inner object within the outer object with this syntax:

```
OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

Inner Classes

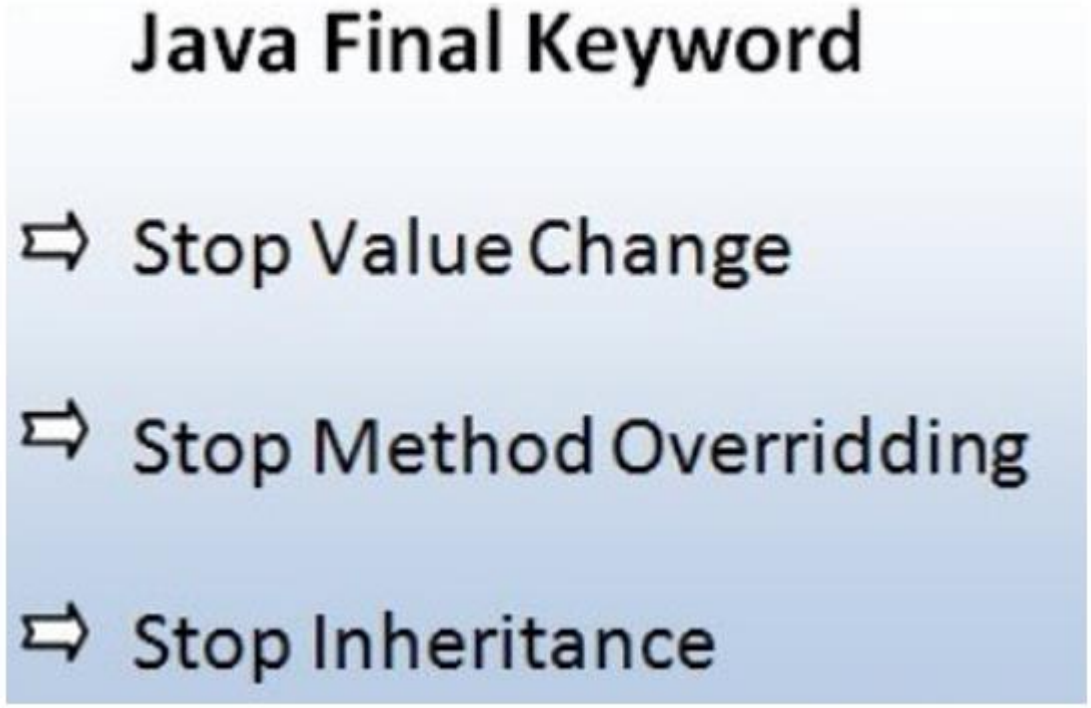
```
class OuterClass {
    static int outer_x = 10;
    int outer_y = 20;
    private int outer_private = 30;
    // inner class
    class InnerClass {
        void display()
        {
            System.out.println("outer_x = " + outer_x);
            System.out.println("outer_y = " + outer_y);
            System.out.println("outer_private = "+ outer_private);
        }}

// Driver class
public class Main {
    public static void main(String[] args)
    {
        // accessing an inner class
        OuterClass outerObject = new OuterClass();

        OuterClass.InnerClass innerObject
            = outerObject.new InnerClass();

        innerObject.display();
    }
}
```

```
outer_x = 10
outer_y = 20
outer_private = 30
```

Comparison between a Normal or Regular Inner class and a static Nested class

S.No.	Normal/Regular inner class	Static nested class
1.	Without an outer class object existing, there cannot be an inner class object. That is, the inner class object is always associated with the outer class object.	Without an outer class object existing, there may be a static nested class object. That is, a static nested class object is not associated with the outer class object.
2.	As the main() method can't be declared, the regular inner class can't be invoked directly from the command prompt.	As the main() method can be declared, the static nested class can be invoked directly from the command prompt.
3.	Both static and non-static members of the outer class can be accessed directly.	Only a static member of an outer class can be accessed directly.

Final Keyword



Example

1) Java final variable

```
class Bike{
final int speedlimit=90;//final variable
void run(){
 speedlimit=400;
}
public static void main(String args[]){
Bike obj=new  Bike();
obj.run();
}
}//end of class
```

Output:

```
Main.java:4: error: cannot assign a value to final variable speedlimit
      speedlimit=400;
      ^
1 error
```

2) Java final method

```
class Bike{
final void run(){System.out.println("running");}
}

class Honda extends Bike{
   void run(){System.out.println("running safely with 100kmph");}

   public static void main(String args[]){
   Honda honda= new Honda();
   honda.run();
   }
}
```

Output:

```
Main.java:6: error: run() in Honda cannot override run() in Bike
       void run(){System.out.println("running safely with 100kmph");}
            ^
  overridden method is final
1 error
```

3) Java final class

```
final class Bike{}

class Honda1 extends Bike{
  void run(){System.out.println("running safely with 100kmph");}

  public static void main(String args[]){
  Honda1 honda= new Honda1();
  honda.run();
  }
}
```

Output:

```
Main.java:3: error: cannot inherit from final Bike
    class Honda1 extends Bike{
                         ^
1 error
```

Final Keyword

Q) Is final method inherited?

Ans) Yes, final method is inherited but you cannot override it.

```
class Bike{
final void run(){System.out.println("running...");}
}
class Honda2 extends Bike{
   public static void main(String args[]){
    new Honda2().run();
   }
}
```

Output:

```
running...
```

Final Keyword

Q) What is final parameter?

If you declare any parameter as final, you cannot change the value of it.

```
class Bike11{
 int cube(final int n){
  n=n+2;//can't be changed as n is final
  n*n*n;
 }
 public static void main(String args[]){
   Bike11 b=new Bike11();
   b.cube(5);
 }
}
```

Output:

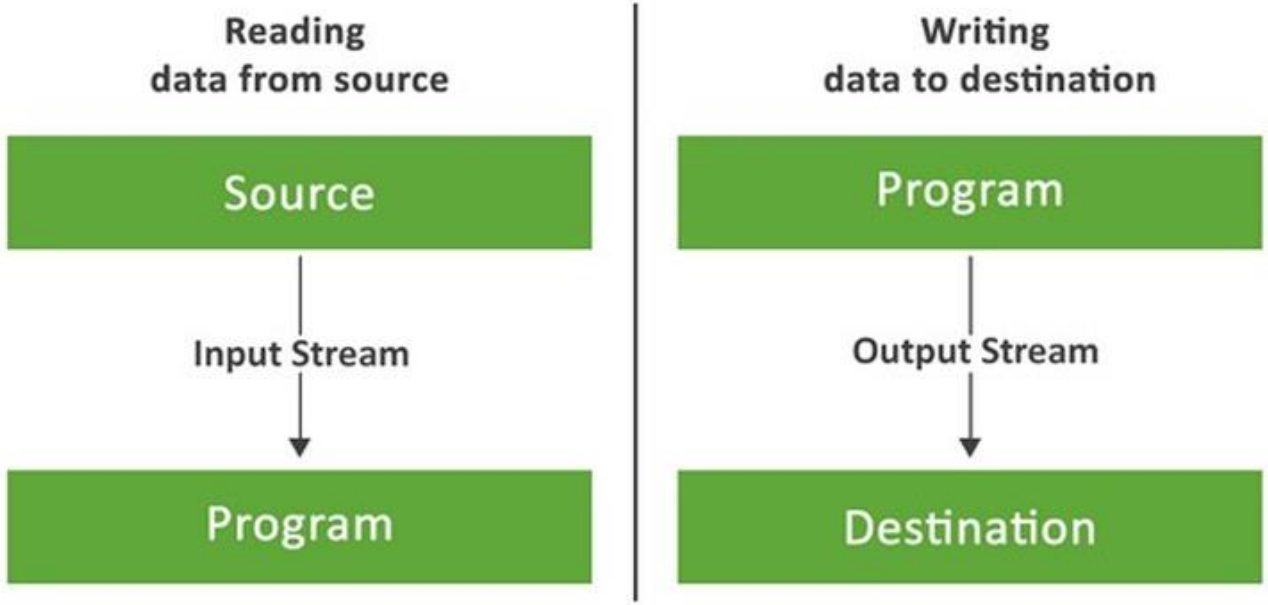
```
Main.java:4: error: not a statement
       n*n*n;
           ^
1 error
```

Unit 1

Stream Classes

Java I/O Basics

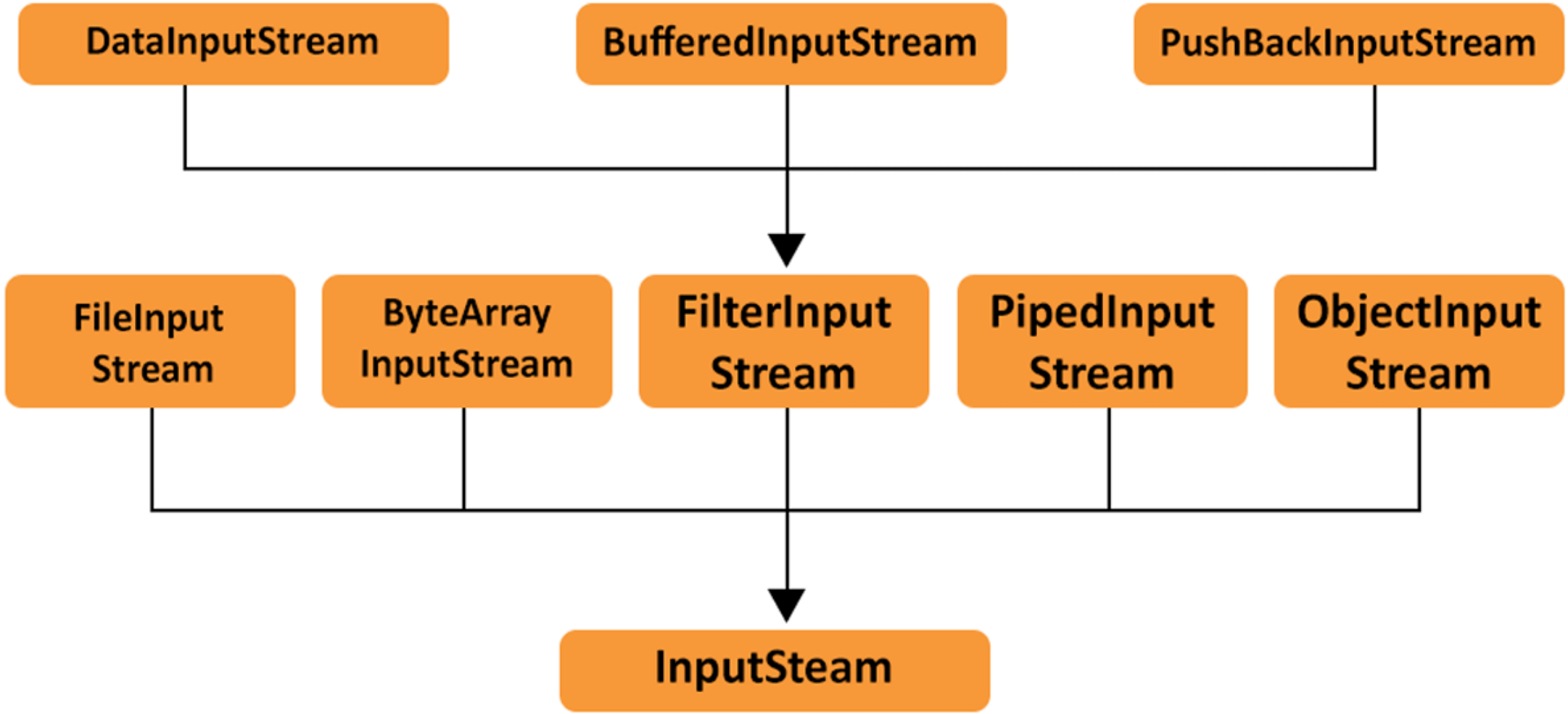
- Java I/O (Input and Output) is used to process the input and produce the output.
- Java uses the concept of a stream to make I/O operations fast.
- The java.io package contains all the classes required for input and output operations. We can perform file handling in Java by Java I/O API.
- The java.io package generally involves reading basic information from a source and writing it to a destination.



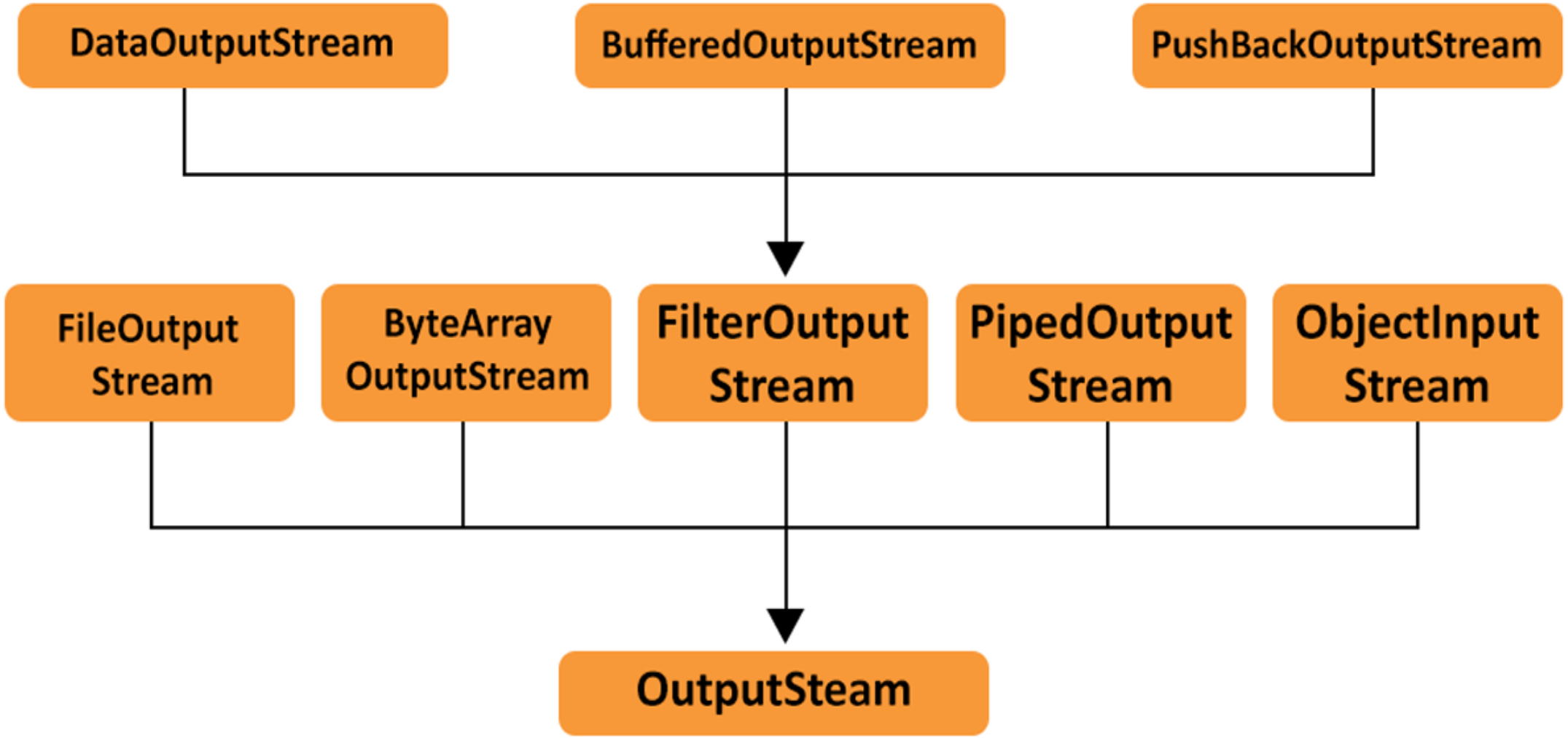
Java IO Streams

- A stream can be described as a series of data.
- Java IO streams are flows of data that a user can either read from or write to.
- Java Streams uses two primary streams to read and write the data.
- These are: InputStream and OutputStream.
- An input stream is used to read the data from a source in a Java application. Data can be anything, a file, an array, a peripheral device, or a socket. In Java, the class java.io.InputStream is the base class for all Java IO input streams.
- An output stream is used to write data (a file, an array, a peripheral device, or a socket) to a destination. In Java, the class java.io.OutputStream is the base class for all Java IO output streams.

Java InputStream



Java OutputStream



Reading Console Input

- In Java, there are four different ways for reading input from the user in the command line environment(console).

1.Using Buffered Reader Class

- This is the Java classical method to take input.
- This method is used by wrapping the System.in (standard input stream) in an InputStreamReader which is wrapped in a BufferedReader, we can read input from the user in the command line.

2. Using Scanner Class

- This is probably the most preferred method to take input.
- The main purpose of the Scanner class is to parse primitive types and strings using regular expressions, however, it is also can be used to read input from the user in the command line.

Reading Console Input

3. Using Console Class

- It has been becoming a preferred way for reading user's input from the command line. In addition, it can be used for reading password-like input without echoing the characters entered by the user; the format string syntax can also be used (like System.out.printf()).

4. Using Command line argument

- Most used user input for competitive coding. The command-line arguments are stored in the String format. The parseInt method of the Integer class converts string argument into Integer. Similarly, for float and others during execution.
- The usage of args[] comes into existence in this input form. The passing of information takes place during the program run. The command line is given to args[]. These programs have to be run on cmd.

Reading Console Input

```
// Java program to demonstrate working of System.console()
// Note that this program does not work on IDEs as
// System.console() may require console
public class Sample {
    public static void main(String[] args)
    {
        // Using Console to input data from user
        String name = System.console().readLine();

        System.out.println("You entered string " + name);
    }
}
```

Writing Console Output

- Writing Console Output in Java refers to writing the output of a Java program to the console or any particular file.
- The methods used for streaming output are defined in the PrintStream class. The methods used for writing console output are print(), println() and write().
- Both print() and println() methods are used to direct the output to the console. These methods are defined in the Print Stream class and are widely used. Both these methods are used with the help of the System.out stream.

The basic differences between print() and println() methods are as follows:

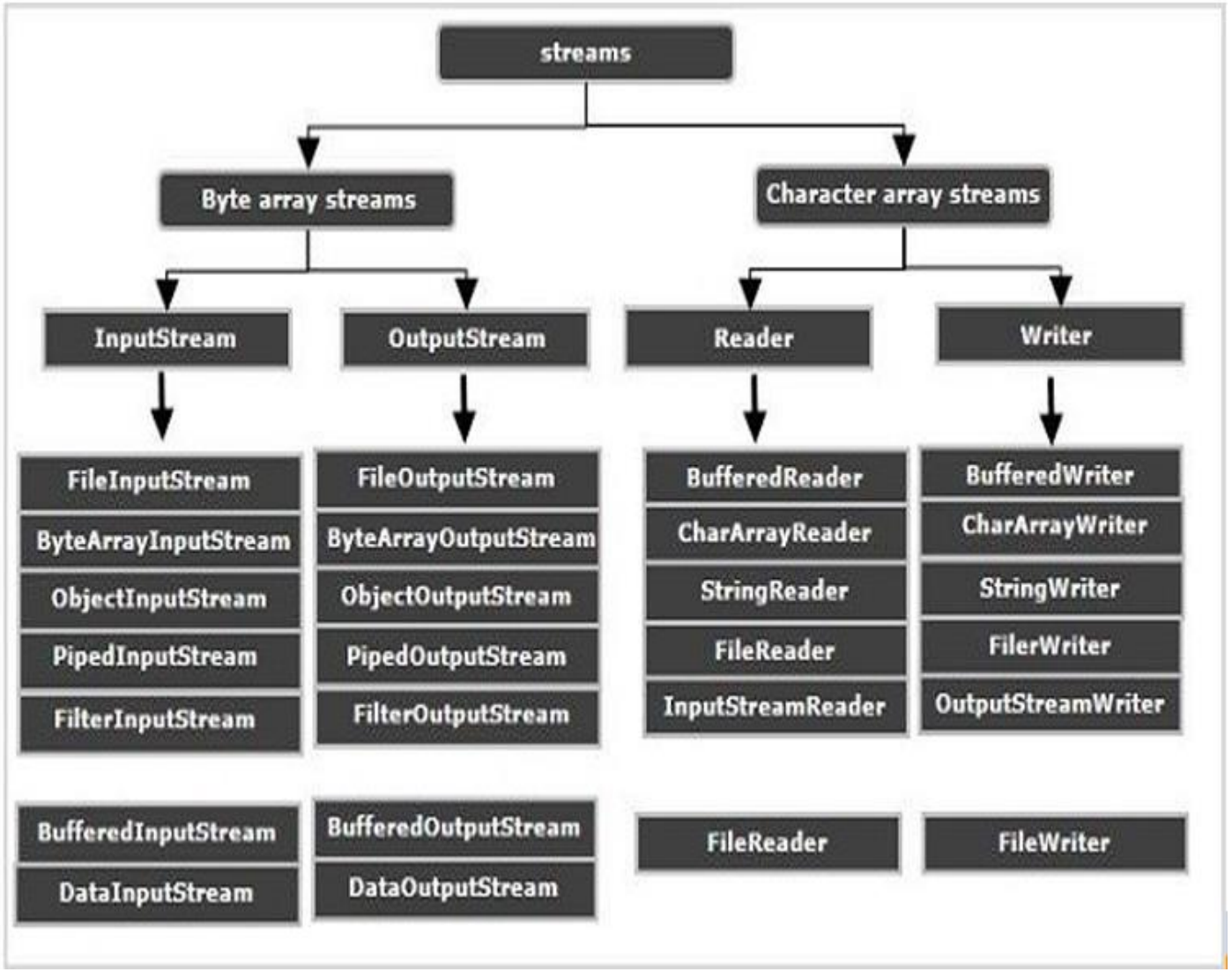
- print() method displays the string in the same line whereas println() method outputs a newline character after its execution.
- print() method is used for directing output to console only whereas println() method is used for directing output to not only console but other sources also.

Writing Console Output Using Write()

```
 class writeEg
{
            public static void main(String args[])
            {
                        int a, b;
                        a = 'Q';
                        b = 65;
                        System.out.write(a);
                        System.out.write('\n');
                        System.out.write(b);
                        System.out.write('\n');
            }
}
```

Types of Streams

- Byte Streams – These handle data in bytes (8 bits) i.e., the byte stream classes read/write data of 8 bits. Using these you can store characters, videos, audios, images etc.
- Character Streams – These handle data in 16 bit Unicode. Using these you can read and write text data only.



Byte Streams FileInputStream

- FileInputStream class is useful to read data from a file in the form of sequence of bytes.

  Constructors of FileInputStream Class

  1. FileInputStream(File file): Creates an input file stream to read from the specified File object.

  2. FileInputStream(FileDescriptor fdobj) :Creates an input file stream to read from the specified file descriptor.

  3. FileInputStream(String name): Creates an input file stream to read from a file with the specified name.

Methods of FileInputStream class

Methods	Action Performed
read()	Reads a byte of data from this input stream
read(*byte[] b*)	Reads up to b.length bytes of data from this input stream into an array of bytes.
read(*byte[] b, int off, int len*)	Reads up to len bytes of data from this input stream into an array of bytes.
available()	Returns an estimate of the number of remaining bytes that can be read (or skipped over) from this input stream.
close()	Closes this file input stream and releases any system resources associated with the stream.

FileOutputStream

- FileOutputStream is an outputstream for writing data/streams of raw bytes to file or storing data to file. FileOutputStream is a subclass of OutputStream.
- To write primitive values into a file, we use FileOutputStream class.
- For writing byte-oriented and character-oriented data, we can use FileOutputStream but for writing character-oriented data, FileWriter is more preferred.

Constructors of FileOutputStream

- 1. FileOutputStream(File file)
- 2. FileOutputStream(File file, boolean append)
- 3. FileOutputStream(String name)
- 4. FileOutputStream(String name, boolean append)

Example

```
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class FileStreamExample {
    public static void main(String[] args) {
        try {
// Create input stream to read file
FileInputStream fis = new
FileInputStream("input.txt");

 // Create output stream to write file
FileOutputStream fos = new
FileOutputStream("output.txt");

int data;
 // Read byte by byte and write to output file
  while ((data = fis.read()) != -1) {
       fos.write(data);
      }
// Close streams
        fis.close();
        fos.close();

System.out.println("File copied successfully!");
}
         catch (IOException e) {
System.out.println("Error occurred: " + e);
        }
    }
}
```

DataInputStream class

- A data input stream enables an application to read primitive Java data types from an underlying input stream in a machine-independent way(instead of raw bytes).
- That is why it is called DataInputStream – because it reads data (numbers) instead of just bytes.
- An application uses a data output stream to write data that can later be read by a data input stream. Data input streams and data output streams represent Unicode strings in a format that is a slight modification of UTF(Unicode Transformation Format)-8.

Constructor

- DataInputStream(InputStream in) Creates a DataInputStream that uses the specified underlying InputStream.

Method

- readUTF(): Reads data from the underlying input stream and converts the bytes into a Unicode string.

DataOutputStream class

- A data output stream lets an application write primitive Java data types to an output stream in a portable way.
- An application can then use a data input stream to read the data back in.

Constructor

- DataOutputStream (OutputStream out): Creates a new data output stream to write data to the specified underlying output stream.

Method

- writeUTF (String str) : Writes a string to the underlying output stream using modified UTF-8 encoding in a machine-independent manner.

Example

```
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class DataStreamExample {
    public static void main(String[] args) {
        try {
            // Writing data to a file
            DataOutputStream dos = new
DataOutputStream(
                    new FileOutputStream("data.txt"));

            dos.writeInt(10);
            dos.writeDouble(25.5);
            dos.writeUTF("Hello Java");
            dos.close();

            // Reading data from the file
            DataInputStream dis = new
DataInputStream(new
FileInputStream("data.txt"));

            int num = dis.readInt();
            double value = dis.readDouble();
            String text = dis.readUTF();

            dis.close();

            System.out.println("Integer: " + num);
            System.out.println("Double: " + value);
            System.out.println("String: " + text);

        } catch (IOException e) {
            System.out.println("Error: " + e);}
    }
}
```

RandomAccessFile class

- RandomAccessFile is a class in Java that provides random access to a file, allowing you to read or write data at any position within the file. It supports both read and write operations and allows you to seek to a specific position in the file.
- You can replace existing parts of a file too. This is not possible with the FileInputStream or FileOutputStream.

Access Modes

- The Java RandomAccessFile supports the following access modes:

Mode	Description
r	Read mode. Calling write methods will result in an IOException.
rw	Read and write mode.
rwd	Read and write mode - synchronously. All updates to file content is written to the disk synchronously.
rws	Read and write mode - synchronously. All updates to file content or meta data is written to the disk synchronously.

RandomAccessFile class

Creating a RandomAccessFile

- Before you can work with the RandomAccessFile class you must instantiate it. Here is how that looks: RandomAccessFile file = new RandomAccessFile("c:\\data\\file.txt", "rw");

Seeking in a RandomAccessFile

- To read or write at a specific location in a RandomAccessFile you must first position the file pointer at the position to read or write.
- This is done using the seek() method.

Example: RandomAccessFile file = new RandomAccessFile("c:\\data\\file.txt", "rw");

file.seek(200);

RandomAccessFile class

Get File Position

- You can obtain the current position of a Java RandomAccessFile using its getFilePointer() method. The current position is the index (offset) of the byte that the RandomAccessFile is currently positioned at.
- Example: long position = file.getFilePointer();

Read Byte from a RandomAccessFile

- Reading a byte from a Java RandomAccessFile is done using its read() method.
- Example: RandomAccessFile file = new RandomAccessFile("c:\\data\\file.txt", "rw");

  int aByte = file.read();
- The read() method reads the byte located a the position in the file currently pointed to by the file pointer in the RandomAccessFile instance.

RandomAccessFile class

Read Array of Bytes from a RandomAccessFile

- It is possible to read an array of bytes from a Java RandomAccessFile.

```
RandomAccessFile randomAccessFile = new RandomAccessFile("data/data.txt", "r");

byte[] dest      = new byte[1024];
int    offset    = 0;
int    length    = 1024;
int    bytesRead = randomAccessFile.read(dest, offset, length);
```

- This example reads a sequence of bytes into the dest byte array passed as parameter to the read() method. The read() method will start reading in the file from the current file position of the RandomAccessFile. The read() method will start writing data into the byte array starting from the array position provided by the offset parameter, and at most the number of bytes provided by the length parameter. This read() method returns the actual number of bytes read.

Example

```
import java.io.RandomAccessFile;
import java.io.IOException;

public class RandomAccessFileExample {
    public static void main(String[] args) {
        try {
// Open file in read and write mode
RandomAccessFile file = new
RandomAccessFile("sample.txt", "rw");

// Write data to file
            file.writeUTF("Hello Java");
            file.writeInt(100);

// Move file pointer to the beginning
            file.seek(0);

// Read data from file
            String text = file.readUTF();
            int number = file.readInt();

// Close file
            file.close();

System.out.println("Text: " + text);
System.out.println("Number: " + number);

        } catch (IOException e) {
            System.out.println("Error: " + e);
        }
    }
}
```

Character Streams

Character streams are primarily used when working with textual data, such as reading from or writing to text files, network connections, or any other data source that deals with characters.

- The two main abstract classes representing character streams in Java are Reader and Writer.
- The Reader class is used for reading characters, while the Writer class is used for writing characters.
- These classes provide several concrete implementations, such as FileReader, FileWriter, BufferedReader, and BufferedWriter, among others.

Java Reader

Java Reader is an abstract class for reading character streams.

- The only methods that a subclass must implement are read(char[], int, int) and close().
- Some of the implementation class are BufferedReader, CharArrayReader, FilterReader, InputStreamReader, PipedReader, StringReader.

  Java FileReader class is used to read data from the file.
- It returns data in byte format like FileInputStream class.
- It is character-oriented class which is used for file handling in java.

Java Writer

Java Writer is an abstract class for writing to character streams.

- The methods that a subclass must implement are write(char[], int, int), flush(), and close().

Java FileWriter class is used to write character-oriented data to a file.

- It is character-oriented class which is used for file handling in java.
- Unlike FileOutputStream class, you don't need to convert string into byte array because it provides method to write string directly.

Java Reader & Writer

```java
import java.io.*;
public class ReaderExample {
  public static void main(String[] args) {
    try {
      Reader reader = new FileReader("file.txt");
      int data = reader.read();
      while (data != -1) {
        System.out.print((char) data);
        data = reader.read();
      }
      reader.close();
    } catch (Exception ex) {
      System.out.println(ex.getMessage());
    }
  }
}
```

```java
import java.io.*;
public class WriterExample {
  public static void main(String[] args) {
    try {
      Writer w = new FileWriter("output.txt");
      String content = "I love my country";
      w.write(content);
      w.close();
      System.out.println("Done");
    } catch (IOException e) {
      e.printStackTrace();
    }
  }
}
```

Java FileReader and FileWriter Class

```
import java.io.FileWriter;
public class FileWriterExample {
   public static void main(String args[]){
       try{
        FileWriter fw=new FileWriter("D:\\testout.txt");
        fw.write("Welcome to javaTpoint.");
        fw.close();
       }catch(Exception e){System.out.println(e);}
       System.out.println("Success...");
   }
}
```

```
import java.io.FileReader;
public class FileReaderExample {
   public static void main(String args[])throws Exception{
        FileReader fr=new FileReader("D:\\testout.txt");
        int i;
        while((i=fr.read())!=-1)
        System.out.print((char)i);
        fr.close();
   }
}
```

Java BufferedReader and BufferedWriter Class

Java BufferedReader class is used to read the text from a character-based input stream.

- It can be used to read data line by line by readLine() method.
- It makes the performance fast.
- It inherits Reader class.

Java BufferedWriter class is used to provide buffering for Writer instances.

- It makes the performance fast.
- It inherits Writer class.
- The buffering characters are used for providing the efficient writing of single arrays, characters, and strings.

Java BufferedReader and BufferedWriter Class

```
import java.io.*;
public class BufferedWriterExample {
public static void main(String[] args) throws Exception {
   FileWriter writer = new FileWriter("D:\\testout.txt");
   BufferedWriter buffer = new BufferedWriter(writer);
   buffer.write("Welcome to javaTpoint.");
   buffer.close();
   System.out.println("Success");
   }
}
```

```
import java.io.*;
public class BufferedReaderExample {
   public static void main(String args[])throws Exception{
       FileReader fr=new FileReader("D:\\testout.txt");
       BufferedReader br=new BufferedReader(fr);

       int i;
       while((i=br.read())!=-1){
       System.out.print((char)i);
       }
       br.close();
       fr.close();
   }
}
```

Java Serialization

Serialization in Java is a mechanism of writing the state of an object into a byte-stream.

- It is mainly used in Hibernate, RMI, JPA, EJB and JMS technologies.
- The reverse operation of serialization is called deserialization where byte-stream is converted into an object.
- The serialization and deserialization process is platform-independent, it means you can serialize an object on one platform and deserialize it on a different platform.
- For serializing the object, we call the writeObject() method of ObjectOutputStream class, and for deserialization we call the readObject() method of ObjectInputStream class.
- We must have to implement the Serializable interface for serializing the object.

Java Serialization- java.io.Serializable interface

- Serializable is a marker interface (has no data member and method).
- It is used to "mark" Java classes so that the objects of these classes may get a certain capability.
- The Serializable interface must be implemented by the class whose object needs to be persisted.
- The String class and all the wrapper classes implement the java.io.Serializable interface by default.

Java Serialization- ObjectOutputStream class

- The ObjectOutputStream class is used to write primitive data types, and Java objects to an OutputStream. Only objects that support the java.io.Serializable interface can be written to streams.

Constructor

1) public ObjectOutputStream(OutputStream out) throws IOException {}	It creates an ObjectOutputStream that writes to the specified OutputStream.

Important Methods

Method	Description
1) public final void writeObject(Object obj) throws IOException {}	It writes the specified object to the ObjectOutputStream.
2) public void flush() throws IOException {}	It flushes the current output stream.
3) public void close() throws IOException {}	It closes the current output stream.

Java Serialization- ObjectInputStream class

- An ObjectInputStream deserializes objects and primitive data written using an ObjectOutputStream.

Constructor

1) public ObjectInputStream(InputStream in) throws IOException {}	It creates an ObjectInputStream that reads from the specified InputStream.

Important Methods

Method	Description
1) public final Object readObject() throws IOException, ClassNotFoundException{}	It reads an object from the input stream.
2) public void close() throws IOException {}	It closes ObjectInputStream.

Java Serialization- Example

- In this example, we are going to serialize the object of Student class from below code. The writeObject() method of ObjectOutputStream class provides the functionality to serialize the object. We are saving the state of the object in the file named f.txt.

Student.java

```
import java.io.Serializable;
public class Student implements Serializable{
 int id;
 String name;
 public Student(int id, String name) {
  this.id = id;
  this.name = name;
 }
}
```

Java Serialization- java.io.Serializable interface

Persist.java

```
import java.io.*;
class Persist{
 public static void main(String args[]){
  try{
  //Creating the object
  Student s1 =new Student(211,"ravi");
  //Creating stream and writing the object
  FileOutputStream fout=new FileOutputStream("f.txt");
  ObjectOutputStream out=new ObjectOutputStream(fout);
  out.writeObject(s1);
  out.flush();
  //closing the stream
  out.close();
  System.out.println("success");
  }catch(Exception e){System.out.println(e);}
 }
}
```

Output

- Success
- File Contents will be

“\AC\ED\00sr\00StudentJ\EC\9A\F1#\B2Uw\
00I\00idL\00namet\00Ljava/lang/String;xp\
00\00\00\D3t\00ravi”

Example of Java Deserialization

Depersist.java

```
import java.io.*;
class Depersist{
 public static void main(String args[]){
  try{
  //Creating stream to read the object
  ObjectInputStream in=new ObjectInputStream(new FileInputStream("f.txt"));
  Student s=(Student)in.readObject();
  //printing the data of the serialized object
  System.out.println(s.id+" "+s.name);
  //closing the stream
  in.close();
  }catch(Exception e){System.out.println(e);}
 }
}
```

Deserialization is the process of reconstructing the object from the serialized state. It is the reverse operation of serialization. Let's see an example where we are reading the data from a deserialized object.

- Output

211 ravi